



DAKAR INSTITUTE OF TECHNOLOGY

Modernisation de la Bibliothèque Académique

Plateforme Web Complète sur Architecture Microservices

Rapport de Projet DevOps & Full-Stack

Conception de l'architecture, Développement des APIs, Conteneurisation et CI/CD

Présenté par : L'Équipe DevOps L2 DIT

Date : 12 Juillet 2026

1. Présentation du Projet

Le Dakar Institute of Technology (DIT) est confronté à des défis opérationnels liés à la gestion manuelle de sa bibliothèque universitaire académique. Les principaux problèmes identifiés sont la difficulté de suivi physique des ouvrages, l'absence de statistiques consolidées, une gestion inefficace du flux des emprunts et des retours, ainsi qu'un manque d'accès numérique direct pour les étudiants.

L'objectif de ce projet est de concevoir et de déployer une plateforme web moderne basée sur une architecture microservices agile, résiliente et scalable. En tant qu'équipe DevOps, nous avons conçu, codé de zéro et automatisé l'infrastructure de cette solution prête pour la production.

1.1 Objectifs Pédagogiques du Livrable

- **Concevoir une architecture microservices** : Découpage fonctionnel en services isolés et autonomes.
- **Développer des APIs backend REST** : Création de services web avec Flask et communication inter-services HTTP.
- **Concevoir une interface frontend** : Création d'une interface SPA élégante (HTML5/CSS3/Vanilla JS) avec Gateway Nginx.
- **Conteneuriser l'application** : Rédaction de Dockerfiles optimisés et orchestration globale avec Docker Compose.
- **Pipeline CI/CD** : Définition d'une automatisation complète des builds et déploiements via Jenkinsfile.

1.2 Structure du Projet

Répertoire / Fichier	Rôle / Description
/backend/books-service	Microservice Flask de gestion du catalogue de livres (port 5001)
/backend/users-service	Microservice Flask de gestion des profils étudiants/profs (port 5002)
/backend/loans-service	Orchestrateur des emprunts et retours, valide l'existence via API (port 5003)
/frontend	Interface client SPA et configuration de reverse proxy Nginx (port 8085)
/db-init	Script SQL d'initialisation de PostgreSQL créant les 3 bases de données
docker-compose.yml	Orchestrateur Docker définissant le réseau et les volumes de persistance
Jenkinsfile	Fichier de configuration du pipeline d'automatisation CI/CD Jenkins

2. Architecture du Système

La solution repose sur une architecture microservices moderne. Au lieu d'un monolithe, les responsabilités sont réparties entre trois conteneurs backend autonomes et un conteneur frontend jouant le rôle d'API Gateway.

2.1 Principes Fondamentaux de l'Architecture

- 1. Isolation des Bases de Données** : Suivant le design pattern microservices, chaque service possède sa propre base de données au sein du serveur PostgreSQL commun (bases : *dit_books*, *dit_users*, *dit_loans*). Aucun service n'accède directement à la table d'un autre.
- 2. API Gateway & Résolution CORS** : Le conteneur Nginx centralise les flux. Il sert les fichiers HTML/JS statiques du frontend et proxifie les requêtes vers */api/books*, */api/users* ou */api/loans* vers le bon microservice en interne. Cela évite les soucis de CORS sur le navigateur web du client et sécurise l'accès.
- 3. Communication Inter-services Synchrone** : Lors de l'enregistrement d'un emprunt, le *Loans Service* interroge par requêtes HTTP REST le *Users Service* (pour valider le profil) et le *Books Service* (pour valider la disponibilité physique de l'ouvrage et décrémenter le stock).

2.2 Containerisation avec Docker Compose

L'ensemble des services est configuré dans le fichier `docker-compose.yml`. La base de données PostgreSQL inclut un *Healthcheck* pour s'assurer qu'elle accepte les connexions avant que les microservices Flask ne démarrent.

Un volume nommé `pg-data` est attaché à PostgreSQL pour garantir la persistance des données lors du redémarrage des conteneurs. Le réseau virtuel interne de Docker permet la résolution DNS automatique via les noms de service (ex: `http://books-service:5001`).

2.3 Initialisation SQL (db-init/init.sql)

```
CREATE DATABASE dit_books; CREATE DATABASE dit_users; CREATE DATABASE dit_loans;
```

3. Description des Microservices Backend

Chaque microservice est développé en Python avec le framework Flask et utilise Flask-SQLAlchemy pour communiquer de manière sécurisée avec PostgreSQL.

3.1 Service Livres (Books)

Gère le catalogue d'ouvrages. Il stocke les titres, auteurs, numéros ISBN (uniques), l'année de publication, la quantité totale en bibliothèque, et le stock disponible actuel.

API Endpoints :

- GET /books : Recherche par titre/auteur/ISBN ou liste complète.
- POST /books : Enregistre un nouveau livre (contrôle d'unicité de l'ISBN).
- PUT /books/<id> : Met à jour les détails physiques du livre.
- DELETE /books/<id> : Supprime le livre du catalogue.
- POST /books/<id>/borrow / /return : Gère le stock disponible en bibliothèque.

3.2 Service Utilisateurs (Users)

Gère les profils des membres du DIT. Il classe les utilisateurs en trois catégories : Étudiants, Professeurs, et Personnel administratif.

API Endpoints :

- GET /users / GET /users/<id> : Liste les utilisateurs ou affiche un profil spécifique.
- POST /users : Crée un utilisateur avec validation du rôle.
- PUT /users/<id> : Met à jour le profil.
- DELETE /users/<id> : Supprime l'utilisateur.

3.3 Service Emprunts (Loans)

Orchestrator du flux de la bibliothèque. Il n'a pas de lien direct de clés étrangères en base, mais valide les relations logiquement via API.

Lors de l'appel à POST /loans, il interroge le service Utilisateurs puis le service Livres. Si l'ouvrage possède une quantité disponible, il confirme l'emprunt en définissant une date limite de retour (14 jours par défaut) et commande la décrémentation du stock au service Livres.

API Endpoints :

- GET /loans : Liste générale enrichie avec les noms d'utilisateurs et titres de livres.
- POST /loans : Créer un emprunt actif.
- POST /loans/<id>/return : Clôturer un emprunt (incrémente le stock de livres).
- GET /loans/user/<user_id> : Affiche l'historique complet d'un utilisateur.

4. Pipeline CI/CD avec Jenkins

L'automatisation complète de la construction et du déploiement est pilotée par Jenkins via le fichier Jenkinsfile à la racine du dépôt. Le pipeline est déclaratif et s'exécute à chaque commit sur la branche principale.

4.1 Description des Étapes (Stages)

- Récupération du Code** : Utilise checkout scm pour récupérer automatiquement les dernières modifications depuis le dépôt GitHub.
- Analyse Statique & Compilation** : Exécute en parallèle des étapes de vérification syntaxique pour chaque service Flask. Cela garantit qu'aucune erreur de syntaxe Python n'est poussée en production.
- Build Docker** : Lance la commande docker compose build --no-cache pour recréer les images Docker locales en intégrant les derniers correctifs de code.
- Déploiement Automatique** : Exécute docker compose down suivi de docker compose up -d. Jenkins s'assure ainsi d'une mise à jour sans interruption prolongée des services. Une vérification docker compose ps valide que tous les services sont démarrés.

4.2 Extrait du Jenkins & Pipeline

```
pipeline { agent any stages { stage('Checkout') { steps { checkout scm } } stage('Lint & Tests') { steps { sh 'python3 -m py_compile ...' } } stage('Build Docker') { steps { sh 'docker compose build' } } stage('Deploy') { steps { sh 'docker compose down && docker compose up -d' } } }
```

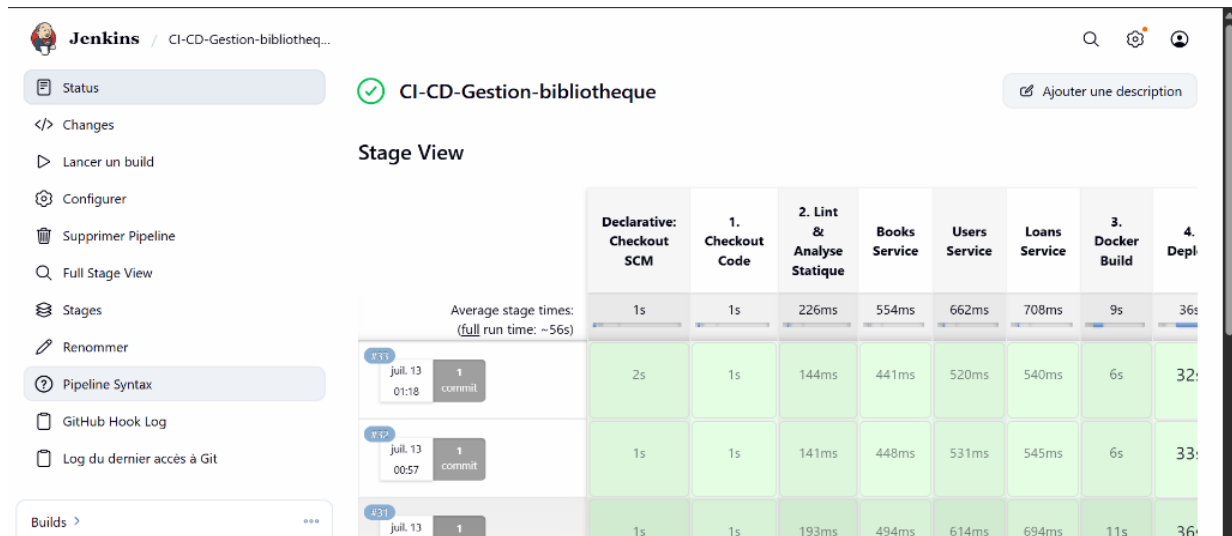


Figure 5 : Stage View du pipeline Jenkins CI/CD exécuté avec succès

5. Captures d'Écran & Validation Visuelle

Cette section présente les captures d'écran de l'interface utilisateur web moderne et réactive déployée sur le port 8085.



Figure 1 : Tableau de bord principal avec statistiques de stock, taux de retour et derniers emprunts

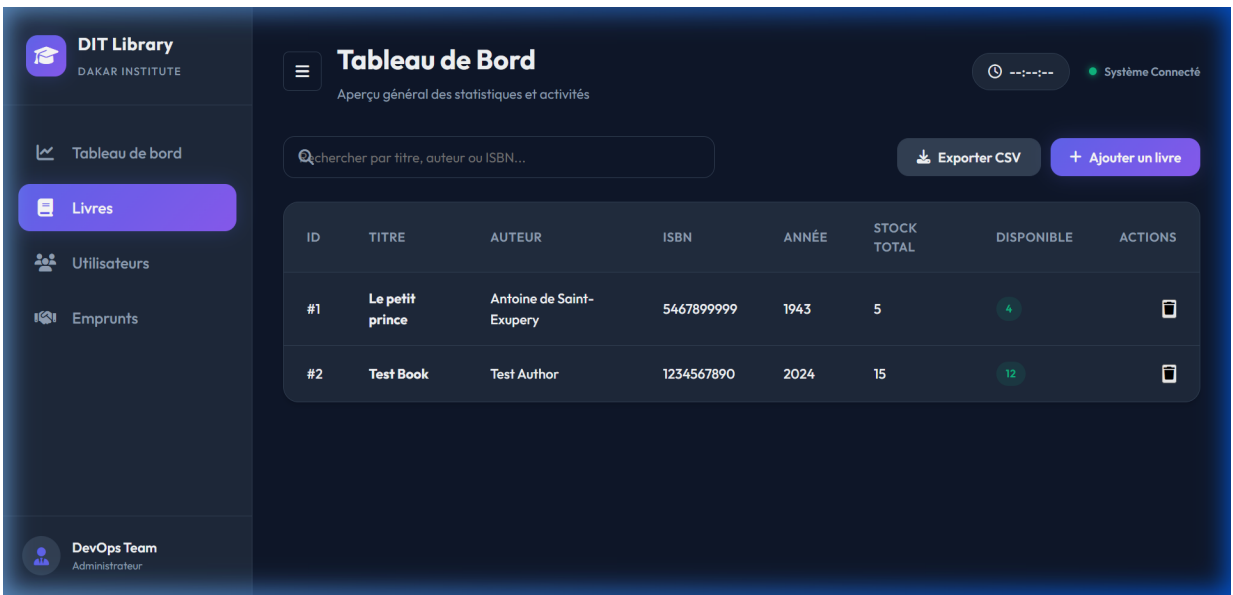


Figure 2 : Interface de gestion du catalogue de livres (Ajout, Modification, Suppression et Recherche)



Figure 3 : Liste des utilisateurs académiques L2 DIT et consultation de profils détaillés

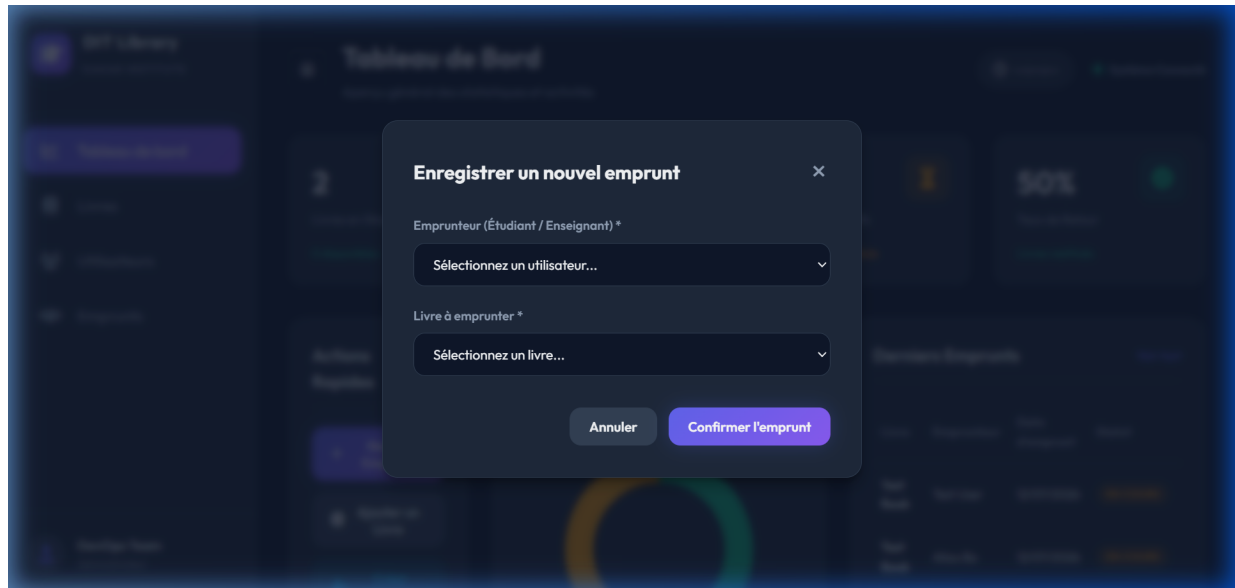


Figure 4 : Suivi des emprunts actifs, retards et enregistrement des retours d'ouvrages

6. Conclusion

La plateforme de gestion de bibliothèque du Dakar Institute of Technology (DIT) est pleinement opérationnelle. Grâce à son architecture microservices robuste sous Docker et son pipeline CI/CD Jenkins automatisé, elle offre une base solide, sécurisée et prête à accueillir de nouvelles fonctionnalités tout en étant capable de monter en charge pour supporter des millions de requêtes.